

빠르게 돌리기 전에, 판정과 위임부터 맞춰야 한다

MoAI의 4단계 지침, 하위 에이전트의 권한과 호출 관계, 관련 규칙 전체를 대조했다. 핵심은 에이전트 수 자체가 아니라 **입력·소유권·실패 판정·재사용 조건**의 불일치다.

기준 HEAD 2213871afb7d · branch main · session 01a08e64-c9d3-76f3-a589-5d5d893e1b62

HTML: explainer / basic · 한국어 기술 보고서 · 생성 기준 2026-09-11T03:20:30.465Z

01. 핵심 판단

검토 항목은 “무엇이 쓰여 있는가”, “실제로 무엇이 재현됐는가”, “아직 무엇을 측정해야 하는가”를 나누어 읽어야 한다. 아래 40개 모두가 실행 시간으로 입증된 병목이라는 뜻은 아니다.

84 / 84

규칙 파일 기계 조사

11 + 23

에이전트 / workflow 파일

18 / 22

High / Medium 검토 항목

9

격리 동작 재현 항목

판단: 병렬 수 확대보다 실패 판정과 캐시 정합성 수정이 먼저다.

신규 파일을 바꿔도 같은 검증 키가 나오고, 실패한 sync 혹은 두 번째 호출은 조용히 종료됐다. lint·병렬 build 예제는 실패를 성공 종료로 바꿨다. 여기에 SPEC 본문 소유권, 작은 Tier의 파일 구성, Git 전달 정책이 맞물려 재감사와 재위임을 유발할 수 있다.

F02 · F03 · F04 · F07 · F08 · F10 · F11 · F17

확인된 사실

현재 파일의 문구·줄·해시, 격리 fixture의 출력, 두 Go 패키지의 선택 테스트 PASS. 모든 규칙의 행수·바이트·frontmatter와 규범문을 기계 검사했다.

아직 확인하지 않은 것

실제 4단계 전체 수행 시간, 모델 토큰/요금, 대기열, native hook의 최종 차단 효과, 원격 보호 정책. 절감률이 나 “몇 배 빨라진다”는 수치는 제시하지 않는다.

용어: 캐시는 이전 검사 결과의 재사용, 키는 그 결과가 현재 입력과 같은지 판별하는 값, fixture는 실제 저장소를 건드리지 않는 작은 시험 환경이다. AC는 기능의 합격 조건, Tier는 작업 규모 분류, fan-out은 여러 독립 작업을 나누어 동시에 호출하

는 방식이다.

02. 실행 관계와 대기 지점

프로젝트 문서가 계획의 입력이 되고, 승인된 계획이 구현의 기준이 되며, 구현 증거가 문서 동기화와 전달의 근거가 된다. 각 경계에서 소유자와 입력 파일이 달라지므로 같은 내용을 다시 만드는 비용이 생기기 쉽다.

1 · project	2 · plan	3 · run	4 · sync
docs → plan-auditor 프로젝트 정보 확정	spec → plan-auditor Tier·AC·구현 승인	spec 재계획 → develop 검사·독립 판정	judge + draft + MX docs → Git 전달
단계	명시된 순서	산출물	주요 연결 문제
project	모드 판별 → 기존 코드 분석/신규 인터뷰 → docs → plan 감사 → 승인 → 선택적 harness	product.md · structure.md · tech.md · harness-spec	docs 작성과 감사의 입력 계약; 신규 질문 4회 분리 F14 · F15 · F26
plan	문맥/명확성 → 연구 → 계획 → Tier → SPEC → 감사 반 복 → 승인/전달	Tier S: spec+plan; M/L: acceptance와 설계/연구 조건부	Tier가 늦음; S 입력 누락; 감사 false block/false pass F01 · F12 · F13 · F16 · F17 · F19
run	plan gate → 문맥/도구 → 재계획/승인 → 작업 분해 → TDD/DDD → 검사/독립 판정 → 전달	코드 · 테스트 · progress Run 증거	재계획·중복 검사·캐시 무효화; owner와 routing 일치 필요 F02 · F03 · F20 · F23 · F25 · F37
sync	preflight → 품질/보안/MX/coverage → 차이 분석 → 문서 → 최종 검사 → Git 전달	README · CHANGELOG · Sync 증거 · 종결 상태	동시 draft의 선행 입력; 중복 full suite; 실패 은폐; status mutation F04 · F05 · F06 · F07 · F10 · F11 · F21 · F30 · F32

병렬화 예시: sync 초안과 judge는 서로의 결과를 읽지 않아도 되지만, 초안은 Phase 11의 divergence report를 요구한다. 따라서 “둘을 동시에 시작”하기 전에 그 공통 입력부터 확보해야 한다. 독립 작업과 선행 입력을 구별하지 않으면 fan-out을 늘려도 대기가 줄지 않는다. F21

03. 11개 에이전트의 책임과 관계

각 에이전트는 자신의 역할 문서를 갖지만, 실제 호출은 workflow가 결정한다. 역할 문서의 금지와 호출부의 요청을 함께 보아야 하며, 역할 이름만으로 실행 가능성을 판단해서는 안 된다.

에이전트 / 정적 설정	단계·입력	산출물·권한 경계	정적 skills / 관련 항목
builder-harness inherit / medium	project 선택 프로젝트 분석·harness-spec	agent/skill/plugin의 메타 구조 본문·업무 코드 작성 제외; docs와 구조/내용 분리	moai-foundation-cc F26 · F28
e2e-tester inherit / low	run/sync UI 조건부 사용자 경로·앱 실행 정보	e2e 스크립트·실행·화면 증거 TDD 단위 테스트 작성과 분리	moai-workflow-testing F31 · F32 · F34
manager-design inherit / high	plan UI 조건부 UI SPEC·디자인 시스템	D1~D5 화면·디자인 전달 구현 코드는 develop; SPEC 본문은 spec	moai-domain-frontend F15 · F25
manager-develop inherit / medium	run 핵심 / sync 수정 시 승인 SPEC·cycle_type·파일 범 위	코드·테스트·progress Run 증거 draft → in-progress; 보안/배포 자문과 분리	moai-foundation-core F08 · F32 · F37
manager-docs inherit / low	project / sync 핵심 project 정보 또는 승인된 구현 차이	project 문서·README·CHANGELOG·Sync 증거 SPEC 본문 금지; sync 단일 commit에서 상 태 종결	moai-foundation-core F07 · F14 · F17 · F18 · F21
manager-git sonnet / low	plan/run/sync 전달 현재 branch·HEAD·증거·승인	push·PR·통합 결과 모든 Tier PR 위임 지침; main-direct와 충돌	moai-foundation-core F08 · F09 · F10 · F11
manager-lead inherit / high	Tier L / kanban / factory 조건 부 milestones·소유 경로 ·dispatch	분할 위임·peer 검증·통합 조율 유일하게 Agent 도구 보유; leaf의 재위임 금 지	moai-foundation-core, moai-workflow- project F22 · F29 · F39
manager-spec inherit / medium	plan 핵심 / run 계획 / sync 되 돌림 요구·연구·Tier·승인	spec/plan/acceptance/design/research 본 문 초기 draft; docs로 본문 소유권 이전 금지	moai-foundation-core, moai-workflow- spec F07 · F16 · F17 · F18 · F25
plan-auditor inherit / high	project/plan 및 run 진입 구현 전 문서·Tier-iteration	독립 PASS/FAIL·review-N·근거 본문 작성자와 분리; project 입력 예외 미연 결	없음 F01 · F03 · F12 · F13 · F14 · F19
super-advisor inherit / high	모든 단계 선택적 좁은 의사결정 질문·근거	비구속 자문·대안 판정·구현·SPEC 작성 권한 없음	moai-foundation-core F24 · F28
sync-auditor inherit / high	run 완료 / sync fallback 구현·AC·검증 증거	독립 PASS/FAIL·부족 증거 사전 SPEC 감사와 충돌; clean sync는 생략 규정	moai-foundation-quality F06 · F15 · F23 · F33

호출 층위: retained 파일은 11개다. `Explore·general-purpose`는 별도 runtime 타입이다. 4dim script는 Explore judge 4개를 만든다. 보안·성능·devops 검토는 `general-purpose`에 역할을 주입하는 경로다. `manager-lead`만 정적 도구 목록에 Agent를 가진다. Tier L의 milestones ≥ 3, files ≥ 10, cross-domain 조건을 함께 만족하는 조율 및 kanban/factory 역할은 조건부다.

안전 경계: 사용자의 답변·승인은 orchestrator가 담당한다. 독립 auditor는 작성자의 자기 판정을 대신 검증하며, docs 작성자가 구현 후 AC를 바꾸어 결과를 맞추어서는 안 된다. model/effort 표는 frontmatter의 관측값이며 실제 호출 시 resolver override와 실행 비용은 측정하지 않았다.

설정·역할 원문을 확인하는 방법

11개 agent의 frontmatter, 헤딩, 규범문을 agent-inventory.json에 보존했다. hooks와 permissionMode는 이 파일에서 확인할 수 있다. 도구 보유 자체를 권한 우회가 실제 발생했다는 증거로 해석하지 않았다. 이번 보고서 작성에는 하위 에이전트를 호출하지 않았다.

04. 40개 상세 검토 항목

High는 실패 은폐·판정/권한 불일치·필수 입력 단절 때문에 관련 경로를 고치기 전 우선 해결을 권고하는 항목이다. Medium은 조건부 반복 비용·범위·설정·관측 개선이다. PR 전체에 대한 merge-blocking 판정은 수행하지 않았다.

원문 발췌의 [...]는 생략을 뜻한다. 전체 경로·명령·stdout·해시는 evidence.json의 동일 ID에서 확인한다.

F01 High 문서 충돌 · plan → run

감사 캐시는 sticky라지만 조회는 오늘 날짜 파일을 지시한다

run 진입 지침은 시간과 무관한 해시 캐시를 선언하면서 오늘 날짜의 보고서를 읽는다. plan-auditor는 최종 review-N 스트림을 재사용 입력으로, 날짜 파일을 기록 전용으로 규정한다. 어제 PASS와 오늘 첫 run을 연결하는 소비 경로가 문서상 일치하지 않는다.

개선과 합격 조건

최종 review-N 보고서의 해시·점수·버전을 단일 조회 함수로 해결하고 날짜 보고서는 이력만으로 사용한다. 날짜가 바뀌고 파일 내용이 같은 fixture에서 새 감사 호출 0회를 확인한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 선결 검토 권고

파일·행과 원문 발췌 (2)

phase-execution.md:45

```
45:[...] oai/reports/plan-audit/<SPEC-ID>-<YYYY-MM-DD>.md` (today's date) and
```

plan-auditor.md:369

```
369:[...] ate consults for skip-eligibility; the run-gate's date-file is a verdict record surface only.
```

추적되지 않은 파일의 내용 변경이 검증 키를 무효화하지 않는다

현재 Key는 HEAD, porcelain-v2, tracked diff만 결합한다. 같은 untracked 경로의 내용을 A에서 B로 바꾼 fixture에서 키가 동일했다. 신규 파일 구현·테스트를 staging 전에 수정하는 경로에서는 오래된 검사 결과가 fresh로 해석될 수 있다.

개선과 합격 조건

검사 대상 untracked 파일의 내용 해시를 포함하고 생성 산출물 제외 규칙을 명시한다. 파일 내용·이름·삭제·ignored 산출물·tracked 변경 각각의 키 불변/변경 시험을 둔다. TTL 단축으로 이 결합을 대체하지 않는다.

근거: fixture 재현 / 전체 workflow 미검증 · 선결 검토 권고 · P07 재현

파일·행과 원문 발췌 (2)

key.go:39

```
39:     porcelain, err := gitOutput(ctx, repoDir, "status", "--porcelain=v2")
```

key.go:43

```
43:     diff, err := gitOutput(ctx, repoDir, "diff", "HEAD")
```

공백 정규화가 실행 의미가 다른 문서까지 같은 키로 만든다

normalizeWhitespace는 모든 공백을 합친다. YAML block의 두 줄 명령과 한 줄 명령이 같은 문자열이 되는 것을 실제 Go 함수로 재현했다. acceptance의 shell·Python·YAML 예제에서는 줄바꿈과 들여쓰기가 의미이므로 단순 서식 변경으로 볼 수 없다.

개선과 합격 조건

원본 바이트 해시를 기본으로 사용한다. 서식 정규화가 꼭 필요하면 코드 블록을 보존하는 Markdown parser를 사용하되 복잡도와 이득을 먼저 측정한다. 서로 다른 실행 의미의 fixture는 반드시 다른 키여야 한다.

근거: fixture 재현 / 전체 workflow 미검증 · 선결 검토 권고 · P06 재현

파일·행과 원문 발췌 (1)

audit_cache.go:134

```
134:     fields := strings.Fields(s)
```

실패한 검사도 once-per-commit 표식으로 다음 호출에서 사라진다

후은 검사 시작 전에 HEAD를 기록한다. go vet/build가 7로 실패한 첫 호출은 block JSON을 냈으나, 같은 HEAD의 두 번째 호출은 검사 없이 빈 stdout으로 종료했다. 현재 fixture에서 확인한 것은 후 출력이며 실제 호스트의 Stop 처리까지 재현한 것은 아니다.

개선과 합격 조건

running·passed·failed 상태와 tree key를 분리한다. 같은 실패 상태는 기존 block을 재표시하고, 중단된 running 상태는 회수 후 재검사한다. 성공 캐시만 조용히 재사용한다.

근거: fixture 재현 / 전체 workflow 미검증 · 선결 검토 권고 · P04 재현

파일·행과 원문 발췌 (2)

sync-phase-quality-gate.sh:174

```
174:# it is unchanged. The SHA is recorded BEFORE the checks run, so a slow/killed run
```

sync-phase-quality-gate.sh:185

```
185:     echo "$HEAD_SHA" > "$SENTINEL_FILE"
```

의존성 취약점 검사 지시와 실제 혹은 manifest diff가 다르다

sync 지침은 manifest 변경 여부와 무관한 취약점 검사를 Stop 혹은 수행한다고 선언한다. 실제 스크립트는 마지막 커밋의 manifest diff 유무만 기록하며, 소스 파일 변화가 없으면 그 전에 종료한다. 설치된 취약점 데이터베이스를 조회한 증거는 이 스크립트에 없다.

개선과 합격 조건

manifest 변경 확인과 취약점 스캔을 서로 다른 검사로 명명한다. lockfile hash·scanner/version·advisory freshness로 스캔을 재사용하고, 실제 실행 명령·출력·누락 언어를 기록한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 선결 검토 권고

파일·행과 원문 발췌 (2)

quality-gates-quality.md:140

```
140:**Dependency manifest audit (always runs, regardless of whether manifest files changed in this
SPEC)** — a SEPARATE, automatic mechanism distinct from the agent-invoked security analysis above:
```

sync-phase-quality-gate.sh:310

```
310:git diff "$DIFF_RANGE" -- $DEPS_MANIFESTS > "$GATE_TMPDIR/deps.diff" 2>&1 || true
```

Critical/High 실패와 High 경고, 실패 후 계속 선택이 공존한다

Phase 7과 sync-auditor는 Critical/High 보안 문제를 FAIL로 정한다. Phase 8은 CRITICAL만 차단하고 HIGH는 경고로 처리한다. CRITICAL에서도 Continue with warning을 제시한다. 같은 취약점이 어느 단계에서 발견되는지에 따라 판정이 달라진다.

개선과 합격 조건

severity와 blocking, 사용자가 승인한 예외를 하나의 판정 계약으로 정의한다. High fixture를 Phase 7·8·auditor에 각각 넣어 동일한 중단 결과와 예외 기록을 검증한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 선결 검토 권고

파일·행과 원문 발췌 (2)

sync-auditor.md:38

```
38: | Security | 25% | OWASP Top 10 compliance | Any Critical/High finding |
```

quality-gates-quality.md:137

```
137:- HIGH findings are reported as warnings in PR description
```

manager-docs에 SPEC 본문 수정을 맡기면서 금지한다

sync Step 2.2.1은 requirements·plan·acceptance 본문을 구현에 맞춰 갱신하도록 한다. 실제 수행 주체 manager-docs는 해당 본문을 수정할 수 없고 manager-spec으로 되돌려야 한다. 이 지시쌍은 거절·재위임을 유발하거나 승인된 기준을 사후 변경하게 한다.

개선과 합격 조건

divergence를 먼저 분류하고 필요한 SPEC 본문 변경만 manager-spec에 넘긴다. 변경된 AC는 독립 재검토 후 구현과 연결하며 docs는 승인된 차이와 frontmatter만 반영한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 선결 검토 권고

파일·행과 원문 발췌 (2)

doc-execution.md:166

```
166:- Update spec.md requirements to reflect actual implementation
```

manager-docs.md:107

```
107:[...] lly implemented, a last-minute AC clarification — this agent MUST return a structured  
blocker report (per `.claude/rules/moai/core/agent-common-protocol.md` § Blocker Report Format)  
and the orchestrator re- [...]
```

작은 Tier의 직접 push와 모든 Tier의 PR 의무가 충돌한다

spec-workflow, run, sync는 Tier S/M에서 main 직접 push를 기본으로 명시한다. manager-git은 모든 Tier에서 push와 PR을 자신에게 위임하도록 한다. 현재 원격 보호 설정은 조회하지 않았으므로 서버에서 거절된다고 단정하지 않는다. 로컬 지침 간 경로 불일치는 확인됐다.

개선과 합격 조건

한 개의 delivery policy를 owner·tier·branch·approval 조건으로 정의하고 모든 호출부가 참조하도록 한다. S/M/L 및 명시적 --pr 조합의 실행계획을 표 기반 시험으로 대조한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 선결 검토 권고

파일·행과 원문 발췌 (2)

spec-workflow.md:25

```
25:- **Route A — Hybrid Trunk main-direct (default; Tier S / Tier M):** manager-develop commits
and pushes directly to `main`; there is NO per-phase PR and NO per-phase branch. Phase transitions
are triggered by c [...]
```

manager-git.md:5

```
5: Invocation gate: invoked for PR creation across ALL tiers (S/M/L) per the PR-mandatory policy
(enforce_admins: true). Tier L uses heavy ceremony (long-lived branch + full CI matrix); Tier S/M
uses light cerem [...]
```

plan 위치·branch switch·reset 지시가 공유 checkout 보호와 충돌한다

spec-workflow는 plan을 main checkout에서 실행하도록 하고 cleanup에 reset --hard를 제시한다. sync는 PR 직후 checkout을 지시한다. main-checkout guard와 이번 AGENTS 계약은 공유 트리의 branch 전환을 금지한다. 지침 실행을 위해 보호를 무시할 근거가 없다.

개선과 합격 조건

launcher로 진입한 worktree에서 전 단계를 유지하고 별도 integration owner가 원격을 처리한다. 공유 checkout에서는 read-only probe와 허용된 명시 경로 commit만 가능하도록 경로 시험을 만든다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 연결 검토 권고

파일·행과 원문 발췌 (3)

spec-workflow.md:50

```
50:- Step 1 (plan) MUST execute in main checkout on BOTH routes. NO L2/L3 worktree at this step.
Plan artifacts are markdown only – no code conflict – and main-authored plans enable cross-SPEC
reference for plan-a [...]
```

delivery.md:297

```
297:**github_flow**: `git checkout {main_branch} && git pull origin {main_branch}`
```

main-checkout-branch-guard.md:16

```
16:[ZONE:Evolvable] [HARD] The orchestrator MUST NOT change branch state in the primary project
checkout. Specifically forbidden there:
```

lint 실패를 도구 미설치로 바꾸고 성공 종료한다

which && golangci-lint || echo SKIP 구조에서 lint가 존재해도 nonzero이면 SKIP이 출력된다. lint=7 fixture는 미설치 문구와 exit 0을 반환했다. 이런 결과를 CI mirror PASS로 소비하면 실패를 숨긴다.

개선과 합격 조건

도구 존재 검사와 실행을 if/else로 분리하고 lint exit code를 보존한다. absent·pass·fail 세 fixture에서 각각 SKIPPED·PASS·FAIL을 확인한다.

근거: fixture 재현 / 전체 workflow 미검증 · 선결 검토 권고 · P02 재현

파일·행과 원문 발췌 (1)

delivery.md:137

```
137:which golangci-lint && golangci-lint run --timeout=5m \
```

cross-build의 인자 없는 wait가 실패를 전달하지 않는다

예제는 여러 build를 &로 시작하고 인자 없는 wait로 끝난다. 자식 하나가 7로 종료해도 wait_status=0을 반환했다. 출력 파일의 존재만으로 각 target 성공을 대신할 수도 없다.

개선과 합격 조건

각 PID와 target을 묶어 개별 wait status를 수집하거나 도구 수준의 독립 호출을 사용한다. 한 target 실패와 여러 target 실패 fixture 모두 전체 FAIL이어야 한다.

근거: fixture 재현 / 전체 workflow 미검증 · 선결 검토 권고 · P03 재현

파일·행과 원문 발췌 (2)

delivery.md:144

```
144:[...] RCH=amd64 CGO_ENABLED=0 go build -ldflags="-s -w" -o /tmp/ci-build-linux-amd64  
./<your-module> &
```

delivery.md:149

```
149:wait
```

reconciliation이 명시된 SPEC도 검증 예제가 BLOCKING으로 만든다

D7 규칙은 retired/superseded 참조에 명시적 reconciliation이 없을 때 차단한다. 제공된 shell은 상태만 읽고 reconciliation 존재를 검사하지 않는다. explicit reconciliation 문장을 넣은 fixture에서도 BLOCKING이 출력됐다.

개선과 합격 조건

참조 ID별 reconciliation 근거를 문서 구조로 파싱한다. 조정 문구 있음/없음, 다른 ID의 문구, 역사 예시를 구분한 fixture로 오탐과 누락을 검증한다.

근거: fixture 재현 / 전체 workflow 미검증 · 선결 검토 권고 · P10 재현

파일·행과 원문 발췌 (2)

plan-auditor.md:328

```
328:[...]     echo "BLOCKING: $SID has status=$STATUS but is referenced without reconciliation"
```

plan-auditor.md:315

```
315:[...] "reversal", "supersede", "absorb", "carve-out") — otherwise BLOCKING
```

다른 절의 build tag가 syscall 검증을 통과시킨다

규칙은 같은 절이나 문단의 build 제약을 요구하나 shell은 문서 전체에서 한 번만 찾는다. syscall 절에는 제약이 없고 다른 절에 //go:build가 있는 fixture가 아무 차단 출력 없이 끝났다.

개선과 합격 조건

각 syscall 사용과 해당 module의 build 제약을 연결한다. 문서에서는 같은 구조 노드의 명시적 연결을 검사하고, 코드가 생기면 해당 target의 cross-build를 별도로 검증한다.

근거: fixture 재현 / 전체 workflow 미검증 · 선결 검토 권고 · P11 재현

파일·행과 원문 발췌 (2)

plan-auditor.md:343

```
343: AC verification), verify nearby (within the same section or paragraph) the
```

plan-auditor.md:357

```
357: if ! grep -qE '//go:build|cross-platform exemption|EXCL.*syscall' <new-spec.md>; then
```

project 문서 감사 호출과 auditor의 spec.md 필수 입력이 맞지 않는다

project는 product-structure-tech를 만든 뒤 plan-auditor에 project 디렉터리를 넘긴다. agent의 입력 계약은 spec.md가 없으면 보고서 없이 AUDIT BLOCKED를 반환한다. project 전용 rubric을 선택하는 분기와 파일 계약이 agent 본문에 제시되지 않았다.

개선과 합격 조건

문서 유형을 typed input으로 전달하고 project용 필수 입력·평가항목·출력 경로를 정의한다. SPEC을 만들지 않은 정상 project fixture로 실제 감사 완료를 확인한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 선결 검토 권고

파일·행과 원문 발췌 (2)

doc-generation.md:49

```
49:[...] nt to audit project documents at .moai/project/ - document type: project, iteration 1."
```

plan-auditor.md:460

```
460:[...] ot found, the agent returns a single-line error: "AUDIT BLOCKED: spec.md not found at {path}" and exits without producing a report.
```

구현 전 sync-auditor 호출이 agent의 역할 제한과 겹친다

project thorough cross-validation, plan thorough SPEC-review, run의 구현 전 계약 협상은 sync-auditor를 사용한다. agent는 post-implementation only와 pre-implementation 금지를 명시하면서 자기 본문에는 계약 협상도 남아 있다.

개선과 합격 조건

계약 제한은 plan-auditor 또는 비판정 자문으로 이동하고 sync-auditor는 구현 증거 판정에 집중한다. 각 호출이 어떤 입력과 권한으로 실행되는지 계약 시험을 둔다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 조건부 개선 후보

파일·행과 원문 발췌 (3)

spec-assembly.md:252

```
252:[...] ue`: after plan-auditor PASS, additionally invoke sync-auditor in SPEC-review mode to
cross-validate must-pass criteria. If sync-auditor disagrees with plan-auditor's PASS, treat as
FAIL and trigger one additio [...]
```

sync-auditor.md:6

```
6: Operates post-implementation only — once code exists and acceptance criteria are testable.
Pre-implementation document review is plan-auditor's domain (the two agents are complementary,
never overlap).
```

sync-auditor.md:122

```
122:Negotiated before implementation in the thorough harness (Phase 10), then carried across
iterations:
```

Tier를 연구·계획·사용자 검토 뒤에 결정한다

연구를 선택한 경로에서는 Phase 60이 research.md를 만들고 Phase 80이 상세 계획을 작성한 뒤 Phase 9에서 Tier를 묻는다. Tier S는 research.md 생략 대상이다. 연구는 권장 단계이므로 항상 발생하는 낭비가 아니라 최종 S인 작업에서의 중복 후보이다.

개선과 합격 조건

요청의 scope와 위험으로 임시 Tier를 먼저 정하고 사용자가 이미 정한 Tier를 재사용한다. 연구 중 근거가 생기면 상향한다. S fixture에서 불필요한 연구 artifact 생성 0회를 측정한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 조건부 개선 후보

파일·행과 원문 발췌 (3)

clarity-interview.md:87

```
87:### Phase 6: Deep Research (Recommended)
```

spec-assembly.md:29

```
29:### Phase 9: Tier Judgment Socratic Question (LEAN Workflow)
```

spec-assembly.md:55

```
55:[...] ec.md` + `plan.md`; AC is inline in `spec.md §3`. No `acceptance.md`, no `design.md`, no  
`research.md`. Phase 11 plan-auditor uses 0.75 PASS threshold.
```

작은 Tier가 생략한 acceptance.md를 하류가 여전히 요구한다

Tier S는 AC를 spec.md에 둔다. 반면 manager-docs의 필수 CHANGELOG self-test는 acceptance.md만 읽으며, plan 완료 목록도 acceptance.md를 일률적으로 요구한다. S의 비용 절감 경로가 하류의 파일 누락 처리와 연결되지 않았다.

개선과 합격 조건

resolveArtifacts(tier)와 resolveACSource(tier)를 단일 계약으로 사용한다. S는 spec inline, M/L은 acceptance를 읽고 누락과 빈 결과를 구분한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 선결 검토 권고

파일·행과 원문 발췌 (3)

spec-assembly.md:55

```
55:- **Tier S** → Phase 10 creates only `spec.md` + `plan.md`; AC is inline in `spec.md` §3. No  
`acceptance.md`, no `design.md`, no `research.md`. Phase 11 plan-auditor uses 0.75 PASS threshold.
```

manager-docs.md:89

```
89:  grep -oE 'AC-([A-Z0-9]+-)*[0-9]+' .moai/specs/<SPEC-ID>/acceptance.md | sort -u | wc -l
```

spec-assembly.md:517

```
517:- Phase 10: All SPEC files created (spec.md, plan.md, acceptance.md, spec-compact.md)
```

허용된 AC 하위 ID a/b를 CHANGELOG 검사에서 하나로 센다

manager-spec은 AC-V3R6-001a/001b를 서로 다른 하위 기준으로 허용한다. manager-docs의 grep은 숫자까지만 추출하여 두 ID를 1개로 계산했다. 통과한 AC 개수를 부정확하게 보고하거나 불필요한 재확인을 유발한다.

개선과 합격 조건

AC 식별자 parser 또는 공용 정규식을 사용한다. suffix·domain-less·digit domain·table/headings의 중복을 구분하여 distinct count를 확인한다.

근거: fixture 재현 / 전체 workflow 미검증 · 조건부 개선 후보 · P12 재현

파일·행과 원문 발췌 (2)

manager-spec.md:139

```
139:[...] ffix to pair sub-criteria within one logical AC (`AC-V3R6-001a` / `AC-V3R6-001b`). That convention applies ONLY inside `acceptance.md` body and is not validated by spec-lint. SPEC IDs themselves MUST NEVER carr [...]
```

manager-docs.md:89

```
89:  grep -oE 'AC-([A-Z0-9]+-)*[0-9]+' .moai/specs/<SPEC-ID>/acceptance.md | sort -u | wc -l
```

Tier 상한과 workflow 반복 지시가 다르다

harness와 auditor는 $S=1/M=2/L=3$ 으로 반복 수를 정한다. plan workflow는 standard/thorough에서 3회, cross-validation 불일치에는 추가 1회를 제시한다. 상한을 어느 계층에서 집행하는지 따라 호출 수가 달라진다.

개선과 합격 조건

remaining_attempts를 orchestrator가 소유하고 모든 reviewer 호출이 같은 예산을 소비하도록 한다. timeout 재시도와 내용 수정 재감사는 별도 원인으로 기록하되 총 상한을 명시한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 조건부 개선 후보

파일·행과 원문 발췌 (3)

harness.yaml:75

```
75: plan_audit_tier_ceilings:
```

spec-assembly.md:252

```
252:[...] grees with plan-auditor's PASS, treat as FAIL and trigger one additional iteration.
```

plan-auditor.md:412

```
412:[...] olved number of times per SPEC. The Tier-resolved ceiling is the SSOT at
`.moai/config/sections/harness.yaml` → `harness.plan_audit_tier_ceilings` (S=1, M=2, L=3; SPEC-
SYNC-PARALLEL-DOCS-001 A6). The agent cons [...]
```

현재 phase 번호와 skip/resume 번호가 혼재한다

minimal skip 목록은 0.5·2.75·2.8a 등을 쓰지만 본문은 1~20으로 실행한다. Phase 1 cache hit 후 Phase 1로 진행, coverage 완료 후 Phase 1로 진행 같은 문구도 남아 있다. resume는 완료 phase를 건너뛰므로 숫자 혼용은 재실행·누락의 원인이 될 수 있다.

개선과 합격 조건

정수 순서와 별개로 안정적인 phase_id를 두고 skip/resume를 ID로 해석한다. 모든 jump 대상 존재, DAG 순환 없음, deprecated ID 매핑을 기계적으로 검사한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 조건부 개선 후보

파일·행과 원문 발췌 (3)

harness.yaml:84

```
84:     skip_phases: [0, 0.5, 2.0, 2.5, 2.75, 2.8a, 2.9, 2.10]
```

phase-execution.md:54

```
54:- Skip Step 3 and proceed to Phase 1.
```

quality-gates-quality.md:317

```
317:- If coverage target met: Proceed to Phase 1
```

Phase 7에서 시작할 초안이 Phase 11 결과를 요구한다

docs drafters는 audit와 같은 Phase 7 turn에 출발해야 한다. 입력은 Phase 11 Step 1.5 divergence report다. entry router는 그 문서를 Phase 11에 on-demand로 읽도록 한다. 입력을 먼저 계산하는 명시적 preflight가 없으면 동시 실행 지시를 그대로 만족할 수 없다.

개선과 합격 조건

diff·SPEC·divergence를 공통 immutable snapshot으로 먼저 만든 뒤 audit와 draft를 함께 시작한다. missing input으로 재위임한 횟수와 draft 폐기율을 기록한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 선결 검토 권고

파일·행과 원문 발췌 (2)

sync.md:67

```
67:[...] RENTLY with the Phase 7-10 audit fan-out — in the SAME turn Phase 7 is entered, NOT
serially after the audit completes. The former scheduling read the Phase Routing Table strictly
top-to-bottom (Phase 7 → Phase [...])
```

doc-execution.md:140

```
140:[...] ds its input from SPEC artifacts + git diff + the Phase 11 Step 1.5 divergence report. A
drafter does NOT read the concurrent audit's quality report, verdict, or per-dimension scores. The
docs draft and the aud [...]
```

5개 docs·4개 judge·MX shard를 동일 turn에 요구한다

문서 초안 5개, 품질 judge 4개, MX shard 3~5개를 동시 시작하도록 한다. 합계 12~14개는 사이트마다 인용하는 3~5 범위와 다르다. 최신 규칙은 3~5를 권고로 바꿨으므로 이를 런타임 hard-cap 위반으로 단정할 수 없지만 전체 배치 예산은 없다.

개선과 합격 조건

한 scheduler가 실제 잔여 슬롯을 읽고 전체 동시 수를 제한한다. MX와 구조 검증을 먼저, 이후 가능한 슬롯에 judge/draft를 넣고 queue 대기과 peak concurrency를 측정한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 조건부 개선 후보

파일·행과 원문 발췌 (3)

doc-execution.md:126

```
126:[...] in parallel before any final artifact is written. Five read-only drafters launch in a
single turn — five is the top of the fanout concurrency ceiling of 3–5 concurrent `Agent()` calls
(`.claude/rules/moai/workf [...]
```

quality-gates-quality.md:162

```
162:[...] or spawns the `FO-SYNC-2` MX shard fan-out in the same single-turn multi-`Agent()` batch
that enters Phase 7, so the MX findings are ready by the time the audit returns. The MX scan is
input-independent (SPEC-S [...]
```

orchestration-mode-selection.md:32

```
32:| `fanout` | N concurrent (3–5 ADVISORY band; hard bound: runtime subagent cap, default 20 —
$C.2) | Multiple `Agent()` invocations in one assistant turn | Multi-domain research that does NOT
meet Agent Teams p [...]
```

run의 4차원 결과와 sync의 판정 소유자가 서로 다르다

run은 같은 fan-out script의 결과를 evidence로만 인정하고 cold sync-auditor 판정을 추가한다. sync는 happy path에서 4dim 결과를 binding으로 승격하며 cold auditor를 생략한다. 서로 다른 목적일 수 있으나 전환 조건과 재사용 계약이 분리되어 있다.

개선과 합격 조건

증거 수집과 판정을 독립 계약으로 정의한다. 동일 tree·AC·rubric에서 재사용 가능한 범위를 명시하고, contested/INCOMPLETE에만 추가 검토한다. 실제 호출 감소는 trace로 확인한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 조건부 개선 후보

파일·행과 원문 발췌 (2)

task-decomposition.md:110

```
110:[...] e user. The script's harmonic-mean aggregate is evidence, not a verdict: the binding
PASS/FAIL remains owned by `sync-auditor`, which may cite the aggregate but is never replaced by
it. Implementation Kicko [...]
```

sync.md:81

```
81:[...] BINDING for the sync-phase quality decision and SHALL NOT spawn the cold `sync-auditor`
subagent. The four parallel xhigh judges subsume the one serial judge on the clean path
(attributable diff-check, not a [...])
```

4차원 judge는 xhigh를 고정하고 model 정책은 xhigh를 배제한다

sync-audit-4dim은 네 judge에 xhigh를 직접 지정한다. model-policy는 profile matrix에서 xhigh를 의도적으로 제외하고 목적별 resolver를 정의한다. 비용 차이는 이번에 측정하지 않았지만 선택 정책이 한 경로에 적용되지 않는 것은 확인된다.

개선과 합격 조건

purpose=verify-judge의 resolved model/effort를 실행 입력으로 전달한다. 동일 fixture에서 profile별 유효 effort와 품질·출력 토큰·호출 횟수를 비교한 뒤 기준을 선택한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 조건부 개선 후보

파일·행과 원문 발췌 (2)

sync-audit-4dim.js:188

```
188:[...] tionality', phase: 'Judge', agentType: 'Explore', effort: 'xhigh', schema: JUDGE_SCHEMA
  }},
```

model-policy.md:157

```
157:`xhigh` is absent from the matrix on purpose: on Opus it scores the same as `high` at
materially higher cost, so it is strictly dominated. `max` is reserved for the two rows whose
invocation is rare AND whose m [...]
```

승인된 계획을 run에서 manager-spec이 다시 분석하고 승인받는다

plan Phase 8과 10이 계획을 확정된 뒤 run Phase 5는 다시 요구·전략을 추출하고 gate-run-1 승인을 받는다. 이어 Phase 6이 task를 분해한다. 변경 없는 재개에서도 어떤 계획 부분을 재사용하는지 명시된 차이 기반 조건이 없다.

개선과 합격 조건

승인된 plan hash와 execution task graph를 전달한다. 불변이면 입력 검증만 하고, 코드·scope 변화가 있는 부분만 재계획한다. 승인 의미를 유지하되 동일 질문 반복은 합친다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 조건부 개선 후보

파일·행과 원문 발췌 (3)

phase-execution.md:263

```
263:## Phase 5: Analysis and Planning
```

phase-execution.md:282

```
282:## Decision Point 1: Plan Approval
```

clarity-interview.md:157

```
157:### Phase 8: SPEC Planning (Required)
```

신규 프로젝트의 네 extended 질문을 별도 호출한다

기존 프로젝트 Stage B는 남은 축을 한 호출에 묶는다. 신규 프로젝트는 네 축 각각을 별도 AskUserQuestion으로 요구한다. 같은 정보 구조에서 호출 수가 최대 4 대 1로 달라진다. 사용자의 실제 대기 시간은 측정하지 않았다.

개선과 합격 조건

신규·기존 모두 미확정 축만 한 batch로 묻는다. 축별 답변 품질과 명시적 empty 처리는 유지한다. 완성된 입력 fixture에서 재질문 수를 세어 확인한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 조건부 개선 후보

파일·행과 원문 발췌 (2)

mode-detection.md:216

```
216:[...] o `harness-spec.yaml` — see `doc-generation.md`). Present each as a separate
AskUserQuestion with up to 4 options:
```

codebase-analysis.md:173

```
173:[...] LL remaining (un-inferred or ambiguous) axes in a single AskUserQuestion call carrying up
to 4 questions — one question per remaining axis, each question with up to 4 options — replacing
the former per-axis sep [...]
```

5개 기술 키워드면 인터뷰를 건너뛰지만 공통 규칙은 의도 완결을 요구한다

plan의 skip 조건은 기술 키워드 5개만으로 성립한다. 범위·성공 조건 없이 기술명만 나열한 요청도 해당될 수 있다. 공통 질문 규칙은 의도 명확성을 조건으로 한다. 키워드 개수는 의미 완결의 기계적 증거가 아니다.

개선과 합격 조건

scope·constraints·acceptance·authorization의 필드 완결도를 검사한다. 기술명이 많고 목표가 없는 fixture와 짧지만 완전한 요청을 함께 검증한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 조건부 개선 후보

파일·행과 원문 발췌 (2)

context-discovery.md:45

```
45:- Input contains 5 or more distinct technical keywords (e.g., framework names, file paths,
function names, domain terms)
```

askuser-protocol.md:67

```
67:[...] **Termination condition**: Rounds continue until intent clarity reaches 100%; the
interview MUST NOT end prematurely
```

paths 없는 규칙 14개가 185,368바이트다

84개 규칙 전체를 읽어 frontmatter를 검사했다. paths 없는 파일은 14개이며 총 185,368바이트다. 이 값은 디스크의 잠재 상시 로딩 표면이며 실제 주입 토큰·캐시 요금은 아니다. kanban·cross-session·handoff 상세가 단일 작업에도 후보로 남는다.

개선과 합격 조건

상시 규칙은 짧은 dispatch 계약만 남기고 상세를 task-triggered companion으로 옮기는 방안을 우선 평가한다. runtime InstructionsLoaded 목록과 token usage로 실제 절감 여부를 검증한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 조건부 개선 후보 · P08 재현

파일·행과 원문 발췌 (2)

rule-authoring.md:14

```
14:1. Rule files under `.claude/rules/` that lack a top-level `paths:` frontmatter key
```

kanban-dispatch.md:9

```
9:## Scope — when this rule is live
```

fetch와 그 결과를 읽는 rev-list를 독립 병렬 검사로 묶는다

공통 pre-spawn 지시는 git fetch와 origin/main...HEAD 비교를 parallel batch에 넣는다. rev-list는 fetch가 갱신하는 ref를 소비하므로 둘은 독립이 아니다. 실제 경합을 재현하지 않았으며, 소스상 데이터 의존을 확인한 항목이다.

개선과 합격 조건

fetch 완료 뒤 rev-list를 수행한다. 세션 목록 조회만 fetch와 병렬화한다. 지연된 fetch fixture에서 최신 remote ref를 읽는지 확인한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 선결 검토 권고

파일·행과 원문 발췌 (2)

agent-common-protocol.md:289

```
289:[...] ing-tree files, the orchestrator MUST execute the following parallel batch and surface any divergence to the user.
```

agent-common-protocol.md:296

```
296:git rev-list --count --left-right origin/main...HEAD
```

읽기 전용 status가 auto-fix·tag 추가보다 뒤에서 종료한다

status는 read-only health check로 정의되지만 Phase 1 auto-fix, Phase 7 critical auto-fix, Phase 9 Add Missing Tags 뒤에 early exit가 위치한다. mode별 mutation guard가 각 단계에 일관되게 제시되지 않아 status의 안전 계약이 불명확하다.

개선과 합격 조건

entry에서 readOnly=true를 전달하고 모든 writer 경로를 차단한다. lint/tag 결함이 있는 fixture의 status 전후 파일 해시와 git diff가 같아야 한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 선결 검토 권고

파일·행과 원문 발췌 (3)

quality-gates-context.md:50

```
50:- status: Read-only health check. Quick project health report with no changes.
```

quality-gates-quality.md:211

```
211:#### Step 0.6.3: Add Missing Tags (Language-Aware)
```

quality-gates-quality.md:267

```
267:Status mode early exit: If mode is "status", display quality report and exit. No further
phases execute.
```

auto sync도 전체 소스 스캔과 coverage test 생성을 요구한다

auto는 changed files only로 정의되지만 Phase 11은 ALL source files를 스캔한다. Phase 10은 전체 coverage에서 P1/P2 gap의 테스트를 생성한다. 기존에 낮았던 unrelated package의 coverage까지 문서 동기화 작업으로 들어올 수 있다.

개선과 합격 조건

auto는 diff와 영향받는 API·문서만, project는 전체로 분리한다. 기존 coverage debt는 따로 보고하고 변경 때문에 생긴 gap만 동일 scope에서 처리한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 조건부 개선 후보

파일·행과 원문 발췌 (3)

quality-gates-context.md:48

```
48:- auto (default): Smart selective sync of changed files only. PR Ready conversion. Daily development workflow.
```

doc-execution.md:40

```
40:Scan ALL source files (not just changed files) for:
```

quality-gates-quality.md:301

```
301:Generate missing tests for P1 and P2 gaps:
```

full suite·coverage·lint·cross-build를 여러 계층이 다시 요청한다

manager-develop의 최종 full suite, sync Phase 1·3, coverage 전후, local CI mirror가 별도 실행을 명시한다. 일부는 snapshot 소비를 도입했지만 Phase 3·coverage에는 같은 계약이 없다. 동일 run 횟수나 비용을 측정한 수치는 아니다.

개선과 합격 조건

tree·command·toolchain·environment·scope를 키로 한 검증 계획을 공유한다. test+coverage를 가능할 때 한 실행으로 묶고, 부하에 민감한 benchmark는 단독 실행한다. 전체 suite는 위험과 CI 정책에 따라 배치한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 조건부 개선 후보

파일·행과 원문 발췌 (3)

manager-develop.md:132

```
132:- Run the COMPLETE test suite (always full, regardless of LARGE_SCALE; memory guard: batches when needed)
```

quality-gates-context.md:158

```
158:- Run full test suite for detected project language
```

delivery.md:133

```
133:go test -race -coverprofile=coverage.out -covermode=atomic ./...
```

문서는 혹·judge의 공유 snapshot 소비를 선언하나 직접 호출 경로가 없다

sync 지침은 Stop hook과 4dim judge가 동일 snapshot을 소비한다고 한다. 혹 전문에는 moai verify 호출이 없고 실제 vet/build를 실행한다. 4dim prompt도 명령 실행을 직접 요구한다. 런타임이 외부에서 증거를 주입하는지는 관찰하지 않았다.

개선과 합격 조건

실제 consumer entry에 조회·miss 실행·record를 연결하고 reuse/reexecute 사유를 출력한다. fresh fixture에서 test 도 구 호출 0회, dirty fixture에서 1회인 spy 검증을 수행한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 조건부 개선 후보 · P09 재현

파일·행과 원문 발췌 (3)

quality-gates-quality.md:43

```
43:[...] orkflows/sync-audit-4dim.js` 4-dimension judges — all consume this single snapshot keyed by
HEAD SHA rather than each independently re-executing `go test` / `golangci-lint` / `go vet` / `go
test -cover`. Concur [...]
```

sync-phase-quality-gate.sh:225

```
225:          run_step go c1 go vet ./...
```

sync-audit-4dim.js:176

```
176:[...] nality: do the acceptance criteria actually hold? Run the test_command; read the ACs;
verify behavior.
```

Kotlin Gradle 파일이 Java로 분류되고 monorepo는 첫 언어만 선택한다

sync diagnostics는 first match wins이고 Java가 build.gradle.kts를 먼저 소비한다. 혹은 detect_language도 같은 파일을 Java로 반환했다. run은 여러 언어를 모두 준비하도록 하므로 단계별 검사 범위가 달라질 수 있다.

개선과 합격 조건

marker별 후보를 수집해 실제 plugin/source suffix로 구분하고 여러 언어를 반환한다. Kotlin-only, Java-only, Go+Node fixture의 검사 목록을 비교한다.

근거: fixture 재현 / 전체 workflow 미검증 · 조건부 개선 후보 · P05 재현

파일·행과 원문 발췌 (2)

quality-gates-quality.md:19

```
19:Check indicator files in priority order (first match wins):
```

sync-phase-quality-gate.sh:62

```
62:     elif [ -f "$root/pom.xml" ] || [ -f "$root/build.gradle" ] || [ -f "$root/build.gradle.kts"
]; then
```

sync의 아무 변경도 없다는 종료 설명이 선행 mutation과 맞지 않는다

Graceful Exit는 어느 decision point에서 취소해도 문서·Git·branch 변화가 없다고 한다. 하지만 앞 단계에서 auto-fix, tag 추가, coverage test 생성, commit을 수행할 수 있다. 백업도 Phase 12에서야 생긴다.

개선과 합격 조건

각 단계에 적용된 변경과 미적용 변경을 기록하고 취소 시 실제 보존 상태를 보고한다. 자동 rollback은 별도 승인·정확한 범위를 요구하며, 읽기 전용 preview와 apply 경계를 앞에 둔다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 조건부 개선 후보

파일·행과 원문 발췌 (2)

delivery.md:399

```
399:- No changes made to documents, Git history, or branch state
```

doc-execution.md:109

```
109:#### Step 2.1: Create Safety Backup
```

전체 docs·SPEC 복사와 non-empty 검사는 비용과 무결성을 혼동한다

sync는 README·docs 전체·SPEC 전체를 매번 복사하고 non-empty directory check를 무결성 검사라 부른다. 파일 하나만 존재해도 디렉터리가 비어 있지 않으므로 원본 전체의 복구 가능성을 증명하지 못한다.

개선과 합격 조건

승인된 변경 대상만 content hash와 상대 경로 목록으로 백업한다. 기존 Git 객체로 복구 가능한 내용은 중복 복사를 줄이고 uncommitted 원본은 별도 보존한다. 파일별 round-trip 복구 시험을 둔다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 조건부 개선 후보

파일·행과 원문 발췌 (2)

doc-execution.md:115

```
115:- Copy critical files: README.md, docs/, .moai/specs/
```

doc-execution.md:116

```
116:- Verify backup integrity (non-empty directory check)
```

정상 RED와 semantic failure 즉시 종단의 구분이 없다

manager-develop은 실패하는 테스트를 먼저 실행해야 한다. run의 자가진단 지침은 test assertion failure를 즉시 human escalation 대상으로 적는다. expected RED가 예외라는 명시적 상태 조건이 없어 정상 개발 사이클도 중단 대상으로 읽힌다.

개선과 합격 조건

expected_RED, regression, tool_failure를 구분하고 RED 허용은 새 AC 테스트와 해당 cycle에만 한정한다. 기존 테스트 regression은 계속 차단한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 조건부 개선 후보

파일·행과 원문 발췌 (2)

manager-develop.md:103

```
103:[...] t (descriptive name, Arrange-Act-Assert pattern), run it and confirm the RED state, then
record the test-case state via TaskUpdate.
```

run.md:187

```
187:[...] ERIFY; Semantic (data race / deadlock / panic / **test assertion failure**) → IMMEDIATE
escalate | `runtime-recovery-doctrine.md` $3 + `ci-autofix-protocol.md` |
```

phase마다 clear하라는 지시와 warm context 유지 지시가 공존한다

spec-workflow Plan token strategy는 완료 후 /clear를 지시한다. context-window와 cache-aware 규칙은 같은 작업의 문맥 축약·resume와 warm context 유지를 우선한다. handoff의 고정 문구와 증거 재로딩은 별도의 비용이며 실제 세션 비용은 이번 에 측정하지 않았다.

개선과 합격 조건

작업 전환·실제 문맥 한계·감사 독립성 필요 여부로 clear를 결정한다. plan→run이 같은 승인 범위이면 간결한 handoff record를 유지하고 중복 로드를 측정한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 조건부 개선 후보

파일·행과 원문 발췌 (3)

spec-workflow.md:180

```
180:- Execute /clear after completion
```

context-window-management.md:41

```
41:Four cheaper moves come first. Each targets a different cause of context growth, so pick by
cause rather than working down the list:
```

cache-aware-execution.md:25

```
25:9. **Weigh session length as a cost axis** [ZONE:Evolvable] [HARD] One long session is cheaper
than several short ones for the same work — every fresh session re-pays the always-loaded prefix
at write price, a [...]
```

team을 허용한 entry와 retired로 서술한 하위 경로가 섞여 있다

run entry와 최신 orchestration rule은 명시적 team을 허용한다. run phase-execution의 scale 설명과 router SKILL Step 3는 agent-team retired/fallback으로 적는다. 같은 요청이 로딩된 문서 위치에 따라 다르게 해석될 수 있다.

개선과 합격 조건

현재 capability와 역사 설명을 분리한다. 하나의 resolver를 두고 explicit team·unsupported runtime·no flag를 fixture로 검증한다. 지원을 단정하기 전에 현재 도구를 확인한다.

근거: 조항·수치 확인 / 실제 지연 미측정 · 조건부 개선 후보

파일·행과 원문 발췌 (3)

context-loading.md:83

```
83:[...] ection silently downgraded with an info log). The current resolver honors `team` as the
experimental Agent Teams mode; this key is no longer emitted at runtime and exists as the
documented retired-era fallback [...]
```

phase-execution.md:245

```
245:Large-change note: agent-team is retired with the Agent Teams static layer. Multi-domain
research fans out via fanout (3-5 concurrent read-only `Agent()` in one turn); coding-heavy
implementation stays serial ( [...]
```

SKILL.md:366

```
366:[...] kflows/<name>.md` for the target subcommand. (The Agent Teams static layer is retired; a
`--team` flag falls back to sub-agent mode per `.claude/rules/moai/workflow/orchestration-mode-
selection.md` - there is n [...]
```

문서의 TRACE PROBE 설명 자체는 실행 trace가 아니다

워크플로우 파일의 MOAI_TRACE_PHASES와 enter/exit는 주식·의사 흐름에 있다. 이번에 실제 project→sync를 실행하거나 토큰·agent 대기·재시도 로그를 수집하지 않았다. 따라서 병목의 발생 가능성과 이미 재현한 결함을 실제 시간 절감률로 환산할 수 없다.

개선과 합격 조건

현재 routing ledger와 verification 기록을 먼저 활용해 stage_start/end, agent parent, input hash, cache hit, retry cause, tool duration을 묶는다. S/M/L·cold/warm·실패 주입별 p50/p95와 호출 수로 개선을 비교한다.

근거: 미측정 범위의 한정 진술 · 조건부 개선 후보

파일·행과 원문 발췌 (2)

project.md:30

```
30:<!-- Activated by MOAI_TRACE_PHASES=1 environment variable -->
```

SKILL.md:38

```
38:[...] ord call is not emitted, the ledger stays empty — an un-recorded dispatch, NOT an opt-in-off no-op. Do not read an empty routing-ledger as 'opt-in disabled'.
```

05. .claude/rules/moai 전수 목록

전체 84개 파일, 15,288행, 1,131,068바이트를 기계 검사했다. “관련 조항 대조”는 이 보고서의 근거로 사용한 파일이며, “목록·규범문 검사”는 의미 전체나 모든 실행 분기를 검증했다는 뜻이 아니다. 경로 제약이 없다는 표시는 frontmatter의 top-level paths 부재를 말한다.

paths 없는 14개 파일은 총 185,368바이트다. 이를 실제 상시 주입 토큰으로 환산하지 않았다. 전체 규칙의 hash·frontmatter·헤딩·규범문을 함께 제공한다. 부록 파일은 원문 추적용이며 이 HTML의 판단과 예시는 오프라인에서도 읽을 수 있다.

규칙 파일 · 연결 범위	행 / bytes	paths	검사 수준 / 관련 항목
NOTICE.md	185	있음	목록·규범문 검사
라이선스·출처	10,910		

규칙 파일 · 연결 범위	행 / bytes	paths	검사 수준 / 관련 항목
core/agent-common-protocol-reference.md 공통 계약	293 27,870	있음	목록·규범문 검사
core/agent-common-protocol.md 공통 계약	370 25,279	없음	관련 조항 대조 F29
core/agent-hooks.md 공통 계약	86 3,799	있음	목록·규범문 검사
core/askuser-protocol-reference.md 공통 계약	202 15,748	있음	목록·규범문 검사
core/askuser-protocol.md 공통 계약	283 21,822	없음	관련 조항 대조 F27
core/glm-web-tooling.md 공통 계약	190 12,168	있음	목록·규범문 검사
core/hooks-system.md 공통 계약	452 34,167	있음	목록·규범문 검사
core/moai-constitution-detail.md 공통 계약	85 6,836	있음	목록·규범문 검사
core/moai-constitution.md 공통 계약	279 15,470	없음	목록·규범문 검사
core/moai-mcp-tools-catalogue.md 공통 계약	113 7,010	있음	목록·규범문 검사
core/moai-mcp-tools.md 공통 계약	99 6,320	없음	목록·규범문 검사
core/native-idiom-and-register-detail.md 공통 계약	37 2,705	있음	목록·규범문 검사
core/native-idiom-and-register.md 공통 계약	52 3,952	없음	목록·규범문 검사
core/output-style-localization-catalogue.md 공통 계약	70 6,722	있음	목록·규범문 검사

규칙 파일 · 연결 범위	행 / bytes	paths	검사 수준 / 관련 항목
core/settings-management.md 공통 계약	338 24,623	있음	목록·규범문 검사
core/verification-claim-integrity-detail.md 공통 계약	110 6,860	있음	목록·규범문 검사
core/verification-claim-integrity.md 공통 계약	71 8,224	없음	목록·규범문 검사
core/zone-registry.md 공통 계약	928 34,970	있음	목록·규범문 검사
design/constitution.md project·plan / UI	422 23,279	있음	목록·규범문 검사
development/agent-authoring.md 규격·라우팅·구현	378 34,859	있음	목록·규범문 검사
development/agent-patterns.md 규격·라우팅·구현	318 17,423	있음	목록·규범문 검사
development/branch-origin-protocol.md 규격·라우팅·구현	72 4,180	있음	목록·규범문 검사
development/coding-standards.md 규격·라우팅·구현	152 11,276	있음	목록·규범문 검사
development/hook-independence.md 규격·라우팅·구현	219 15,242	있음	목록·규범문 검사
development/karpathy-quickref.md 규격·라우팅·구현	53 2,068	있음	목록·규범문 검사
development/manager-develop-prompt-template.md 규격·라우팅·구현	261 16,144	있음	목록·규범문 검사
development/model-policy.md 규격·라우팅·구현	236 33,954	있음	관련 조항 대조 F24
development/orchestrator-templates.md 규격·라우팅·구현	290 12,885	있음	목록·규범문 검사

규칙 파일 · 연결 범위	행 / bytes	paths	검사 수준 / 관련 항목
development/prompting-best-practices.md 규격·라우팅·구현	78 9,269	있음	목록·규범문 검사
development/rule-authoring.md 규격·라우팅·구현	55 4,062	있음	관련 조항 대조 F28
development/skill-ab-testing.md 규격·라우팅·구현	221 7,809	있음	목록·규범문 검사
development/skill-authoring.md 규격·라우팅·구현	376 24,484	있음	목록·규범문 검사
development/skill-writing-craft.md 규격·라우팅·구현	143 5,751	있음	목록·규범문 검사
development/spec-frontmatter-schema.md 규격·라우팅·구현	241 23,317	있음	목록·규범문 검사
development/sprint-round-naming.md 규격·라우팅·구현	167 11,843	있음	목록·규범문 검사
development/verification-completeness.md 규격·라우팅·구현	168 10,783	있음	목록·규범문 검사
languages/cpp.md run·sync / 해당 언어	109 7,679	있음	목록·규범문 검사
languages/csharp.md run·sync / 해당 언어	100 5,882	있음	목록·규범문 검사
languages/elixir.md run·sync / 해당 언어	111 6,602	있음	목록·규범문 검사
languages/flutter.md run·sync / 해당 언어	112 7,261	있음	목록·규범문 검사
languages/go.md run·sync / 해당 언어	134 4,559	있음	목록·규범문 검사
languages/java.md run·sync / 해당 언어	135 7,592	있음	목록·규범문 검사

규칙 파일 · 연결 범위	행 / bytes	paths	검사 수준 / 관련 항목
languages/javascript.md run:sync / 해당 언어	149 7,945	있음	목록·규범문 검사
languages/kotlin.md run:sync / 해당 언어	123 6,491	있음	목록·규범문 검사
languages/php.md run:sync / 해당 언어	101 3,287	있음	목록·규범문 검사
languages/python.md run:sync / 해당 언어	127 4,278	있음	목록·규범문 검사
languages/r.md run:sync / 해당 언어	156 7,761	있음	목록·규범문 검사
languages/ruby.md run:sync / 해당 언어	158 8,081	있음	목록·규범문 검사
languages/rust.md run:sync / 해당 언어	140 6,177	있음	목록·규범문 검사
languages/scala.md run:sync / 해당 언어	130 4,875	있음	목록·규범문 검사
languages/swift.md run:sync / 해당 언어	122 5,616	있음	목록·규범문 검사
languages/typescript.md run:sync / 해당 언어	135 7,524	있음	목록·규범문 검사
quality/boundary-verification.md plan:run:sync	209 9,309	있음	목록·규범문 검사
workflow/archived-agent-rejection.md 단계·세션·전달	180 17,662	있음	목록·규범문 검사
workflow/cache-aware-execution-reference.md 단계·세션·전달	51 5,881	있음	목록·규범문 검사
workflow/cache-aware-execution.md 단계·세션·전달	45 6,980	없음	관련 조항 대조 F38

규칙 파일 · 연결 범위	행 / bytes	paths	검사 수준 / 관련 항목
workflow/cadence-bridge.md 단계·세션·전달	114 13,615	있음	목록·규범문 검사
workflow/ci-autofix-protocol.md 단계·세션·전달	184 7,037	있음	목록·규범문 검사
workflow/context-window-management-detail.md 단계·세션·전달	119 7,548	있음	목록·규범문 검사
workflow/context-window-management.md 단계·세션·전달	126 8,882	없음	관련 조항 대조 F38
workflow/cross-session-messaging-detail.md 단계·세션·전달	64 7,680	있음	목록·규범문 검사
workflow/cross-session-messaging.md 단계·세션·전달	154 15,852	없음	목록·규범문 검사
workflow/dynamic-workflows.md 단계·세션·전달	180 23,355	있음	목록·규범문 검사
workflow/goal-directive-detail.md 단계·세션·전달	96 19,228	있음	목록·규범문 검사
workflow/goal-directive.md 단계·세션·전달	51 6,600	없음	목록·규범문 검사
workflow/kanban-dispatch-detail.md 단계·세션·전달	256 33,961	있음	목록·규범문 검사
workflow/kanban-dispatch.md 단계·세션·전달	255 32,800	없음	관련 조항 대조 F28
workflow/main-checkout-branch-guard-detail.md 단계·세션·전달	131 8,471	있음	목록·규범문 검사
workflow/main-checkout-branch-guard.md 단계·세션·전달	114 6,395	없음	관련 조항 대조 F09
workflow/moai-memory.md 단계·세션·전달	179 10,890	있음	목록·규범문 검사

규칙 파일 · 연결 범위	행 / bytes	paths	검사 수준 / 관련 항목
workflow/mx-tag-protocol.md 단계·세션·전달	207 7,306	있음	목록·규범문 검사
workflow/native-invocation-model.md 단계·세션·전달	104 15,746	있음	목록·규범문 검사
workflow/nav-tokens.md 단계·세션·전달	118 4,693	있음	목록·규범문 검사
workflow/orchestration-mode-selection.md 단계·세션·전달	290 38,296	있음	관련 조항 대조 F22
workflow/runtime-recovery-doctrine.md 단계·세션·전달	136 18,014	있음	목록·규범문 검사
workflow/session-handoff-examples.md 단계·세션·전달	378 40,449	있음	목록·규범문 검사
workflow/session-handoff.md 단계·세션·전달	174 21,197	없음	목록·규범문 검사
workflow/skill-routing-detail.md 단계·세션·전달	25 1,359	있음	목록·규범문 검사
workflow/skill-routing.md 단계·세션·전달	60 5,595	없음	목록·규범문 검사
workflow/spec-workflow.md 단계·세션·전달	455 38,623	있음	관련 조항 대조 F08 · F09 · F38
workflow/verification-batch-pattern.md 단계·세션·전달	78 5,613	있음	목록·규범문 검사
workflow/worktree-integration.md 단계·세션·전달	480 40,029	있음	목록·규범문 검사
workflow/worktree-state-guard.md 단계·세션·전달	140 6,309	있음	목록·규범문 검사

언어 규칙 16개는 run/sync의 언어 선택과 검사 범위에 연결된다. 디자인 규칙은 UI 작업에서, session·goal·kanban 규칙은 해당 모드에서, 저작/규격 규칙은 메타 산출물을 만들 때 연결된다. 이번에는 언어별 최신 API나 외부 도구 버전의 사실성을 전수 검증하지 않았다.

06. 개선 순서와 성공 기준

시간을 줄이는 변경이 기존 검증을 약화시키면 개선으로 계산하지 않는다. 먼저 잘못된 재사용과 실패 은폐를 막고, 그다음 같은 입력을 반복 소비하는 경로를 줄인다. 아래는 제안이며 아직 구현하지 않았다.

PHASE A runtime/verify-hook-auditor 지침 소유자

판정 신뢰성부터

이 보고서의 실패 fixture를 회귀 테스트로 편입한다. 실패→PASS 변환과 semantic hash collision을 먼저 막는다.

F02 · F03 · F04 · F10 · F11 · F12 · F13

PHASE B orchestrator·SPEC·Git 정책 소유자

권한·입력 계약 통일

tier/phase/mode/owner를 단일 표로 정의한다. 안전 기준을 낮추지 않고 모순된 호출부만 정렬한다.

F01 · F05 · F06 · F07 · F08 · F09 · F14 · F17 · F20 · F29 · F30

PHASE C workflow·scheduler·검증 소비자 소유자

재실행·대기 줄이기

불변 입력 snapshot을 먼저 만들고 자원 한도 안에서 독립 작업만 병렬화한다. 검사별 재사용 이유를 기록한다.

F16 · F19 · F21 · F22 · F23 · F24 · F25 · F26 · F31 · F32 · F33 · F34 · F36

PHASE D harness·관측·문서 소유자

문맥·재개·실측

상시 문맥을 조건부 상세로 분리할지 실측한다. 안정 phase ID와 실제 변경 기록을 사용하고 cold/warm 기준을 비교한다.

F15 · F18 · F27 · F28 · F35 · F37 · F38 · F39 · F40

측정 축	수집 값	비교 조건·합격 판단
정확성	오탐·누락, false PASS, stale cache reuse	이번 실패 fixture 전부 기대 상태로 변경; 기존 정상 fixture 유지
재사용	검사별 hit/miss·재실행 사유·tree key	같은 입력의 중복 실행 감소; 변경된 입력은 반드시 무효화
호출·대기	agent spawn·retry·queue·peak concurrency	S/M/L, cold/warm 분리; 실제 runtime slot 안에서 비교

측정 축	수집 값	비교 조건·합격 판단
사용자 질문	승인 의미·질문 batch 수·동일 질문 재등장	권한 승인은 유지; 이미 완결된 입력의 재질문은 제거
검사 자원	tool duration·full suite 횟수·CPU 경합	동일 machine/toolchain/scope 기준; p50/p95와 실패 주입을 함께 비교
문맥 비용	실제 loaded rules·input/cache/output tokens	문서 bytes 대신 런타임 usage 사용; 안전 규칙 누락 없어야 함

가장 작은 설계: 거대한 새 coordinator를 만들기보다 기존 resolver·verify snapshot·routing ledger를 먼저 연결한다. 단일 정책 표, 공동 입력 snapshot, 안정적인 phase ID, 소비자별 회귀 테스트가 우선이다.

07. Evidence — 명령과 관측 출력

아래는 이번 세션에서 수집한 출력이다. source-derived probe는 현재 함수/예제를 추출한 작은 재현이며 실제 네 단계 실행과 다르다. P01과 P07은 같은 현상의 입력 수준·함수 수준 검증으로, 서로 다른 두 결함으로 세지 않았다.

선택 Go 테스트 · exit 0

```
unset CLAUDE_PROJECT_DIR CLAUDE_SESSION_ID MOAI_SKIP_PLAN_AUDIT MOAI_AUDIT_GATE_T0 && go test
./internal/runtime ./internal/verify -run 'Test.*
(Cache|Hash|Snapshot|Key|Fresh|Skip|RecordCheckConcurrent)' -count=1
```

ok	github.com/modu-ai/moai-adk/internal/runtime	0.476s
ok	github.com/modu-ai/moai-adk/internal/verify	4.992s

Go 1.26.4 darwin/arm64. 위 시간은 명령의 실제 출력일 뿐 MoAI workflow 성능 측정이 아니다. 기존 테스트 통과와 신규 결함 재현은 동시에 성립한다.

P01 · git status --porcelain=v2; git diff HEAD

```
git status --porcelain=v2; git diff HEAD - untracked new.go content A → B in isolated fixture

porcelain_before=? new.go
porcelain_after=? new.go
diff_before=""
diff_after=""
```

```
key_inputs_equal=true
computed_digest_equal=true
```

P02 · bash -c: which() { return 0; }; golangci-lint() { return 7; }; which golangci-lint && golangci-lint run --timeout=5m || echo "SKIP: golangci-lint not installed"

```
bash -c: which() { return 0; }; golangci-lint() { return 7; }; which golangci-lint && golangci-lint run --timeout=5m || echo "SKIP: golangci-lint not installed"
```

```
exit=0
stdout=SKIP: golangci-lint not installed
stderr=
```

P03 · bash -c: (exit 7) & (exit 0) & wait; printf "wait_status=%s" "\$?"

```
bash -c: (exit 7) & (exit 0) & wait; printf "wait_status=%s" "$?"
```

```
exit=0
stdout=wait_status=0
stderr=
```

P04 · bash <source>/sync-phase-quality-gate.sh, twice at identical fixture HEAD; fake go exits 7

```
bash <source>/sync-phase-quality-gate.sh, twice at identical fixture HEAD; fake go exits 7
```

```
{
  "first": {
    "exit": 0,
    "stdout": "{\\hookSpecificOutput\\":
{\\hookEventName\\":\\"Stop\\",\\"decision\\":\\"block\\",\\"reason\\":\\"go vet
failed\\",\\"systemMessage\\":\\"sync-phase quality gate BLOCKED: go vet failed (go vet=7 go build=7
deps_modified=1). Detail: .moai/logs/sync-quality-gate.log\\"}",
    "stderr": ""
```

```

    },
    "second": {
        "exit": 0,
        "stdout": "",
        "stderr": ""
    },
    "tool_calls": [
        "vet ./...",
        "build ./..."
    ]
}

```

P05 · source sync-phase-quality-gate.sh; detect_language <fixture with build.gradle.kts>

```
source sync-phase-quality-gate.sh; detect_language <fixture with build.gradle.kts>
```

```

exit=0
stdout=java
stderr=

```

P06 · go run normalize.go

```
go run normalize.go -- normalizeWhitespace function extracted verbatim from
internal/runtime/audit_cache.go
```

```

exit=0
stdout=different_source=true
normalized_equal=true
normalized="command: | echo first echo second"
stderr=

```

P07 · go run key.go <fixture>

```
go run key.go <fixture> -- Key unchanged; only package/import and error-format adapter for
standalone execution
```

```
exit=0
stdout=key_before=1c2af144e463ac5df7c9afc57cd82e847352f26c:a0de81d7346c1cb9
key_after=1c2af144e463ac5df7c9afc57cd82e847352f26c:a0de81d7346c1cb9
key_equal=true errors=<nil>,<nil>
stderr=
```

P08 · Read every rule file; count paths frontmatter absence and UTF-8 bytes

Read every rule file; count paths frontmatter absence and UTF-8 bytes

```
{
  "files": 84,
  "lines": 15288,
  "bytes": 1131068,
  "no_paths_files": 14,
  "no_paths_bytes": 185368
}
```

P09 · Check literal snapshot consumer symbols in full hook and workflow source

Check literal snapshot consumer symbols in full hook and workflow source

```
{".claude/hooks/moai/sync-phase-quality-gate.sh":
{"verify_cli":false,"snapshot_term":false},".claude/workflows/sync-audit-4dim.js":
{"verify_cli":false,"snapshot_term":true}}
```

P10 · D7 verification verb extracted verbatim; fixture explicitly reconciles superseded SPEC-OLD-001

D7 verification verb extracted verbatim; fixture explicitly reconciles superseded SPEC-OLD-001

```
exit=0
stdout=BLOCKING: SPEC-OLD-001 has status=superseded but is referenced without reconciliation
stderr=
```

P11 · D8 verification verb; syscall and build-tag occur in different sections

```
D8 verification verb; syscall and build-tag occur in different sections
```

```
exit=0
stdout=
stderr=
```

P12 · grep -oE AC-([A-Z0-9]+-)*[0-9]+ ac.txt | sort -u | wc -l; two suffix-distinct AC IDs

```
grep -oE AC-([A-Z0-9]+-)*[0-9]+ ac.txt | sort -u | wc -l; two suffix-distinct AC IDs
```

```
exit=0
stdout=1
stderr=
```

재현 스크립트는 reproductions/에 함께 보관했다. 내부 절대 scratch 경로를 포함하므로 재실행 시 새 격리 디렉터리로 바뀌어야 한다. 실제 저장소에 서 실패 명령을 실행하지 않는다.

08. Claim · Baseline · Gaps · Residual risk

Claim — 이 보고서가 주장하는 것

문서/소스 대조와 격리 동작 재현을 근거로 40개 검토 항목을 제시했다. 84개 규칙의 전수 기계 조사와 핵심 관련 조항의 의미 대조를 수행했다. 모든 의미적 결함을 빠짐없이 찾았다는 보장은 아니다.

Baseline-attribution — 어디에서 측정했는가

/Users/goos/MoAI/moai-adk-go

최초 HEAD 2213871afb7d655f411c46a34fd38bb8153278fc

최종 HEAD 2213871afb7d655f411c46a34fd38bb8153278fc

157개 파일 SHA-256 재검사: 변경 0개. 기존 파일의 변경은 관측되지 않았다.

dirty 작업 트리를 그대로 읽었다. 이미 변경된 설정과 untracked 규칙도 현재 표면으로 포함했다. 제품 소스·지침·설정 수정하지 않았고, commit/push/PR도 실행하지 않았다. 설치된 moai binary는 이 소스 HEAD와 빌드 기준이 다르다.

Gaps — 관찰하지 않은 범위

실제 project→sync 전체 실행, native Stop JSON 처리, 외부 취약점 DB 조회, 현재 원격 branch 보호, 실제 모델 가격·토큰·대기/실행 시간, 모든 언어별 검사, template/mirror 전체 동등성. 전수 규칙 파일 검사는 모든 규칙의 모든 분기 실행과 동의어가 아니다.

Residual-risk — 남는 위험

157개 해시가 같아도 범위 밖 파일·원격 상태·동시 세션은 달라질 수 있다. 문서 모순의 실제 해석은 harness와 호출 입력에 달려 있다. D7/D8 예제의 오탐·누락을 재현했으나 LLM의 최종 판정은 재현하지 않았다. 권한·독립 감사·보안·접근성 검사를 비용 절감 명목으로 제거해서는 안 된다.

직접 확인할 세 가지

① F02와 P07에서 untracked 내용 변경과 같은 키를 대조한다. ② F07/F08의 양쪽 원문을 읽어 owner를 하나로 정한다. ③ 실제 S/M/L trace를 수집한 뒤에만 병렬 수·검사 재사용·문맥 분리가 빨라졌다고 판단한다.

화면·오프라인·인쇄 검증 기록

영실 · MoAI instruction & workflow audit · 2026-09-11

Markdown · 전체 증거 · 규칙 전수 데이터 · 에이전트 전수 데이터